

AI-powered healthcare
assistant that helps hospital
staff access information from
multiple sources through a
single chat interface

MediAssist AI Enterprise Healthcare Agentic AI Copilot

Gaurav Korekar

Project Overview:

The goal of this project is to build an AI-powered healthcare assistant that helps hospital staff access information from multiple sources through a single chat interface.

The system combines Retrieval-Augmented Generation (RAG), Multi-Agent AI, Multimodal AI, MCP-based database access, and cloud deployment to provide accurate and contextual healthcare information.

Problem Statement:

Hospitals keep information in many different places, such as SOPs, policies, patient records, appointment systems, billing systems, and medical reports.

Because the information is spread across multiple systems, finding the right data can take a lot of time and effort.

The goal is to build a single AI assistant that can connect to all these sources and provide fast, accurate, and relevant answers whenever needed.

Project Objectives

The system should be able to:

- Answer questions from healthcare documents
- Retrieve patient-related information from databases
- Analyze uploaded medical reports and prescription images
- Provide healthcare-related recommendations
- Support operational and analytical queries
- Use AI agents to perform different tasks automatically
- Ensure responses are grounded and reliable

Users:

User Type	Purpose
Doctor	View patient history, reports, and recommendations
Front Desk Staff	Check appointments and patient details
Insurance Team	Verify insurance and billing information
Hospital Administrator	Monitor operations and analytics
Clinical Operations Team	Access workflows and procedures

Data Provided / Data Source:

Available Data Source / Documents	File Formats	Used for
Hospital SOPs, Clinical workflow PDFs, Insurance policy documents, Healthcare compliance guidelines, Discharge summaries, Operational manuals, Escalation workflows	PDF files, DOCX documents, TXT documents	RAG pipelines, Vector indexing, Semantic retrieval, Context grounding
Scanned prescriptions Sample medical report images Lab report screenshots	Images	Vision-language understanding, Information extraction, Report understanding, Multimodal reasoning
PostgreSQL - Database Schema: Patients, Admissions, Appointments, Doctors, Departments, Billing, Lab Results, Prescriptions, Insurance Policies, Hospital Branches	-	Analytics Support: To answer analytical questions

Functional Requirements

- Users should be able to ask healthcare-related questions through a chat interface.
- Users should be able to upload PDF documents for information retrieval.
- Users should be able to upload prescription and medical report images for analysis.
- The system should retrieve information from enterprise documents using RAG.
- The system should retrieve structured data through MCP-enabled PostgreSQL tools.
- The system should generate grounded responses with citations and context.
- The system should evaluate responses for relevance and hallucination risks.

Requirement Understanding / Proposed Workflow:

- **User** interacts through the Streamlit application by uploading medical reports, prescription images, or asking questions.
- The **Planner Agent** understands the request, breaks it into tasks, and selects the required agents and tools.
- The **Multimodal Agent** analyses reports and images to extract key medical information such as medicines, diagnoses, observations, and dates.
- The **Retriever Agent** searches enterprise documents using vector databases and retrieves relevant information through semantic search.
- The **MCP Agent** accesses structured data from PostgreSQL systems, such as patient history, appointments, billing, and lab records.
- The **Reasoning Agent** combines information from documents, databases, user queries, and medical reports to generate accurate summaries and recommendations.
- The **Evaluation Agent** checks response quality, grounding, relevance, and hallucination risks using LLM-as-a-Judge evaluation to ensure reliable answers.

Project Modules:

1. **Data Ingestion:** PDF/image ingestion, chunking, embeddings, and metadata creation.
2. **Enterprise RAG:** Vector indexing, semantic/hybrid search, and grounded retrieval using FAISS/ChromaDB.
3. **Multi-Agent System:** Build agents (Planner, Retriever, MCP, Multimodal, Reasoning, Evaluation) using LangGraph.
4. **MCP Server:** Create MCP tools with PostgreSQL connectors and secure tool workflows.
5. **Multimodal AI:** Enable vision-based medical image and report understanding.
6. **Evaluation & Observability:** LLM-as-a-Judge, hallucination checks, and system monitoring.
7. **Guardrails:** Add safety for prompt injection, data leakage, unsafe outputs, and tool misuse.
8. **Dockerization:** Build Docker setup with Dockerfile and Docker Compose.
9. **Azure Deployment:** Deploy Streamlit + FastAPI app using Docker on Azure Container Apps.

Frontend Features (Streamlit)

- AI chatbot with streaming responses for real-time interaction.
- PDF upload for SOPs, policies, and reports.
- Image upload for prescriptions and lab reports.
- Evaluation dashboard showing retrieval traces and agent workflow.

Technologies Used

Streamlit, FastAPI, LangChain, LangGraph, OpenAI/Groq, FAISS/ChromaDB, BLIP/LLaVA, PostgreSQL, MCP, Docker, Azure.

Non-Functional Requirements

The solution should support modular and scalable architecture.

Responses should be generated quickly.

Sensitive data and credentials should be stored securely.

The system should provide logging, monitoring, and observability.

Deployment should be containerized using Docker.

The application should be deployable on Azure cloud infrastructure.

Deliverables:

1. **GitHub Repository** – Complete source code with documentation
2. **Architecture Documentation** – System design, agent workflow, MCP integration, and deployment flow.
3. **Working Demo** – Demonstration of all major workflows
4. **Azure Deployment** – Fully deployed application on Azure cloud.
5. **Evaluation Report** – Includes retrieval quality, hallucination checks, latency, agent performance, and LLM-as-a-Judge results.